Step-by-Step Guide: Setting Up CPU-Based Inference Gateway on GKE

This guide walks you through setting up a fully functional, CPU-based **Inference Gateway** on Google Kubernetes Engine (GKE) using the Kubernetes Gateway API Inference Extension and a vLLM CPU model server.

Overview

This setup deploys an L7 External Application Load Balancer that uses an **Endpoint Picker (EPP)** extension to intelligently route generative AI traffic to a pool of CPU-based vLLM model servers.

Prerequisites

- A GCP Project with the following APIs enabled:
  - Compute Engine API
  - Kubernetes Engine API
  - Network Services API
- `gcloud`, `kubectl`, and `helm` (v3+) installed.
- A **Proxy-only subnet** configured in your target region (Required for GKE L7 Regional Load Balancers).

Step 1: Create the GKE Cluster

Create a standard GKE cluster with the Gateway API enabled. For CPU inference, use nodes with sufficient memory headroom (e.g., `e2-standard-16` or `e2-highmem-16`).

Shell

```
gcloud container clusters create cpu-igw-poc \
    --project=YOUR_PROJECT_ID \
    --zone=us-east5-a \
    --machine-type=e2-standard-16 \
    --num-nodes=3 \
    --gateway-api=standard
```

Get credentials for your cluster:

```shell
gcloud container clusters get-credentials cpu-igw-poc --project=YOUR_PROJECT_ID
--zone=us-east5-a
```

Step 2: Install Inference Extension CRDs

Install the core custom resource definitions required by the Inference Gateway architecture.

```shell
kubectl apply -k
https://github.com/kubernetes-sigs/gateway-api-inference-extension/config/crd
```

Step 3: Deploy the GKE Inference Gateway

Deploy the Gateway resource targeting the Regional External Managed load balancer class.

Create a file named `gateway.yaml`:

```none
kind: Gateway
apiVersion: gateway.networking.k8s.io/v1
metadata:
  name: inference-gateway
spec:
  gatewayClassName: gke-l7-regional-external-managed
  listeners:
  - name: http
    port: 80
    protocol: HTTP
```

Apply the Gateway:

```shell
kubectl apply -f gateway.yaml
```

Wait for the Gateway to receive an external IP address (this may take 3-5 minutes):

```
Shell
kubectl get gateway inference-gateway
```

*Note down the **ADDRESS** returned.*

Step 4: Deploy the CPU-Based vLLM Model Server

Deploy your vLLM model server using the official CPU image.

> [!IMPORTANT] **CRITICAL Memory Adjustments Needed for vLLM on CPU**: By default, the `vllm-openai-cpu` image ignores container memory limits and allocates KV cache based on the *total host node memory*, often causing immediate OS **Out-Of-Memory (OOM)** kills. To resolve this:
>
> 1. Set the **`VLLM_CPU_KVCACHE_SPACE`** environment variable to cap the cache size (e.G., `"4"` GB).
> 2. Ensure container Memory Limits and `/dev/shm` requests are right-sized for your model.

Create `cpu-deployment.Yaml` using a lightweight model like `Qwen2.5-1.5B-Instruct` (which fits in standard RAM and doesn't require a gated token):

```
None
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vllm-cpu-server
spec:
  replicas: 2
  selector:
    matchLabels:
      app: vllm-cpu-server
  template:
    metadata:
      labels:
        app: vllm-cpu-server
        inference.Networking.K8s.Io/engine-type: vllm
    spec:
      containers:
```

```
- name: vllm
  image: "vllm/vllm-openai-cpu:latest"
  imagePullPolicy: Always
  command: ["python3", "-m", "vllm.Entrypoints.Openai.Api_server"]
  args:
  - "--model"
  - "Qwen/Qwen2.5-1.5B-Instruct"
  - "--port"
  - "8000"
  - "--max-model-len"
  - "4096"
  env:
    - name: PORT
      value: "8000"
    # Cap the CPU KV Cache to avoid OOMKilled errors
    - name: VLLM_CPU_KVCACHE_SPACE
      value: "4"
  ports:
    - containerPort: 8000
      name: http
      protocol: TCP
  livenessProbe:
    failureThreshold: 240
    httpGet:
      path: /health
      port: http
      scheme: HTTP
    initialDelaySeconds: 5
    periodSeconds: 5
    successThreshold: 1
    timeoutSeconds: 1
  readinessProbe:
    failureThreshold: 600
    httpGet:
      path: /health
      port: http
      scheme: HTTP
    initialDelaySeconds: 5
    periodSeconds: 5
    successThreshold: 1
    timeoutSeconds: 1
  resources:
     limits:
       cpu: "4"
       memory: "20Gi"
       ephemeral-storage: "10Gi"
     requests:
       cpu: "4"
```

```
            memory: "20Gi"
            ephemeral-storage: "10Gi"
        volumeMounts:
          - mountPath: /dev/shm
            name: shm
      volumes:
        - name: shm
          emptyDir:
            medium: Memory
            sizeLimit: 4Gi
      restartPolicy: Always
```

Apply the deployment:

Shell
```
kubectl apply -f cpu-deployment.yaml
```

Step 5: Deploy InferencePool and EPP using Helm

Use Helm to install the `InferencePool` resource and deploy the Endpoint Picker (EPP) extension, which automatically creates the matching `HTTPRoute`.

Shell
```
export INFERENCE_POOL_NAME=vllm-cpu-server
export GATEWAY_PROVIDER=gke
export MODEL_SERVER=vllm
export MODEL_SERVER_PROTOCOL=http
export IGW_CHART_VERSION=v0

helm install ${INFERENCE_POOL_NAME} \
  --set inferencePool.ModelServers.MatchLabels.App=${INFERENCE_POOL_NAME} \
  --set provider.Name=${GATEWAY_PROVIDER} \
  --set inferencePool.ModelServerType=${MODEL_SERVER} \
  --set inferencePool.ModelServerProtocol=${MODEL_SERVER_PROTOCOL} \
  --set experimentalHttpRoute.Enabled=true \
  --version ${IGW_CHART_VERSION} \

oci://us-central1-docker.pkg.dev/k8s-staging-images/gateway-api-inference-extensio
n/charts/inferencepool
```

Step 6: Verify and Test

Check that both the EPP and Model Server pods are **`Running`** and report **`READY 1/1`**:

```
Shell
kubectl get pods
```

(Note: CPU model compilation/warmup on initial startup can take up to 3-5 minutes).

Verify the HTTPRoute references the InferencePool correctly:

```
Shell
kubectl describe httproute ${INFERENCE_POOL_NAME}
```

Once READY, send a test inference request to the Gateway IP you noted in Step 3:

```
Shell
export GATEWAY_IP="YOUR_GATEWAY_IP"

curl -i http://${GATEWAY_IP}:80/v1/completions \
-H 'Content-Type: application/json' \
-d '{
"model": "Qwen/Qwen2.5-1.5B-Instruct",
"prompt": "Write as if you were a critic: San Francisco",
"max_tokens": 50,
"temperature": 0
}'
```

Cleanup

To delete all resources created during this guide:

```
Shell
helm uninstall ${INFERENCE_POOL_NAME}
kubectl delete -f cpu-deployment.yaml
kubectl delete -f gateway.yaml
```

```
kubectl delete -k
https://github.com/kubernetes-sigs/gateway-api-inference-extension/config/crd
```